

Mini-Project: Socratic AI Tutor

Course: DAT566/DIT408 – Introduction to Data Science and AI

Topic: Retrieval-Augmented Generation for Educational Assistants

Task: RAG-based tutor with hybrid retrieval, grounding checks, and evaluation

Group 77

Team Members

Name:	Mina Tahmasebi Berjouei
Program:	Information and Communication Technology, MSc
Email:	Berjouei@chalmers.se
Hours:	20

Name:	Olga Ratuszniak
Program:	Data Science and AI, MSc
Email:	olgara@chalmers.se
Hours:	20

March 20, 2026

Contents

1	Introduction	2
2	Motivation and Objectives	2
3	Relation to Course Concepts	2
4	System Design and Methodology	3
4.1	Architecture Overview	3
4.2	Ingestion Pipeline	3
4.3	Retrieval and Answer Pipeline	3
4.4	Persistence	4
5	Implementation	4
5.1	Hybrid Reranker	4
5.2	Grounding Check	5
5.3	Supported File Formats	5
6	Evaluation Framework	6
6.1	Golden Dataset	6
6.2	Metrics	6
7	Discussion: Mimicking vs. Understanding	6
8	Known Limitations	7
9	Conclusion	7
	Appendix: Key Implementation Components	8

1. Introduction

This report presents a mini-project that builds a Socratic AI Tutor, an educational virtual assistant based on Retrieval-Augmented Generation (RAG). The system allows users to upload study files, store them as vector embeddings in a persistent database, and ask grounded questions that are answered using evidence retrieved directly from those files.

The project relates to the course concept of **conversational AI and chatbots**, and extends it in three concrete ways: by grounding responses in user-provided documents, by implementing a hybrid retrieval and reranking strategy, and by providing a quantitative evaluation framework for both retrieval quality and answer quality.

2. Motivation and Objectives

Large language models (LLMs) are prone to hallucination: generating plausible but factually incorrect answers. In an educational context this is particularly harmful. RAG addresses this by constraining the model to answer only from retrieved evidence, making responses verifiable and grounded.

Main Objectives

- Provide accurate, evidence-grounded answers from user-provided study files.
- Maintain persistent memory across application restarts.
- Allow users to manage the knowledge base: save, delete, or reset files.
- Evaluate retrieval and generation quality with measurable metrics.

3. Relation to Course Concepts

The project directly extends the course topic of **retrieval-augmented generation and conversational agents**. Standard RAG pipelines retrieve documents and pass them to an LLM. This project extends that baseline in three ways:

1. **Hybrid reranking**: retrieved chunks are reranked using a combination of semantic rank (MMR) and lexical overlap score, improving retrieval precision.
2. **Grounding checks**: before calling the LLM, a grounding strength score is computed between the query and retrieved context. If the score is below a threshold, the system refuses to answer rather than hallucinate.
3. **Evaluation framework**: a golden dataset with Hit@k, MRR, Token F1, and Semantic Similarity metrics provides quantitative feedback on both retrieval and generation quality, going beyond simple manual inspection.

The project also touches on the course discussion of **mimicking vs. understanding**: the tutor retrieves and reranks text but does not truly comprehend it. This distinction is discussed further in Section 7.

4. System Design and Methodology

4.1. Architecture Overview

The application is structured as three UI panels built with Streamlit:

- **Left panel:** knowledge base management (upload, save, list, delete, reset).
- **Center panel:** chat interface for user interaction with the tutor.
- **Right panel:** evaluation results and assessment.

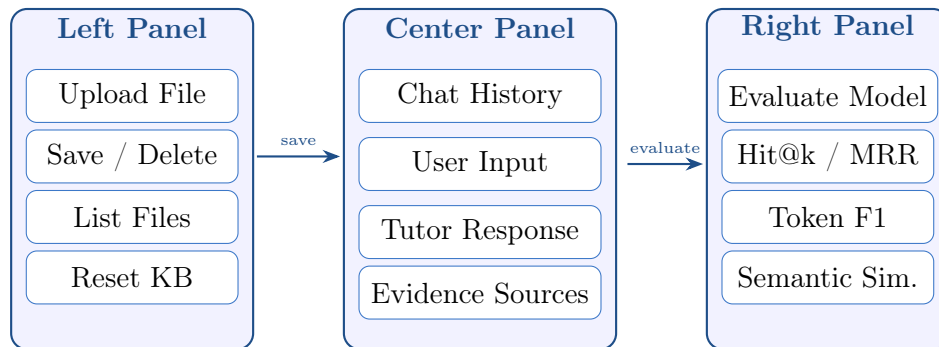


Figure 1: Three-panel Streamlit UI architecture.

File	Responsibility
<code>app.py</code>	Streamlit UI, session state, panel layout, button logic.
<code>database.py</code>	File parsing, chunking, embedding ingestion, hybrid retrieval, delete/reset.
<code>tutor_engine.py</code>	OpenAI calls, chitchat detection, grounding checks, fallback behavior.
<code>evaluation.py</code>	Golden dataset loading, Hit@k, MRR, Token F1, Semantic Similarity.
<code>config.py</code>	Runtime constants and environment variable mapping.
<code>docker-compose.yml</code> / <code>Dockerfile</code>	Containerized deployment with named volumes.

4.2. Ingestion Pipeline

1. User uploads a file (`.txt`, `.md`, `.pdf`, `.csv`, `.json`).
2. The file is parsed into clean plain text using format-specific readers.
3. Text is split into overlapping chunks using a recursive character splitter (chunk size: 450, overlap: 80).
4. Chunks are embedded with `sentence-transformers/all-MiniLM-L6-v2` and stored in ChromaDB with stable IDs to prevent duplicate insertion.

4.3. Retrieval and Answer Pipeline

1. User sends a message.

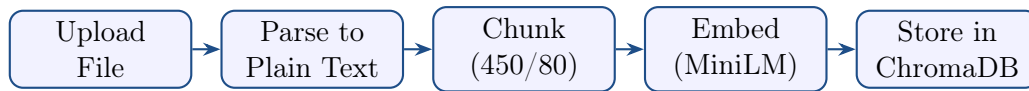


Figure 2: Document ingestion pipeline.

2. If the message is chitchat (greeting, thanks, goodbye), it is answered directly without retrieval.
3. Otherwise, top-k chunks are retrieved using MMR (Maximal Marginal Relevance) and reranked with a hybrid score: $0.7 \times \text{semantic rank} + 0.3 \times \text{lexical overlap}$.
4. A grounding strength score is computed. If below 0.18, the system declines to answer rather than risk hallucination.
5. An extractive answer is attempted first. If successful, it is returned directly. Otherwise, the full context and chat history are sent to the OpenAI API.
6. The answer and retrieved evidence metadata (source, page, chunk) are returned.

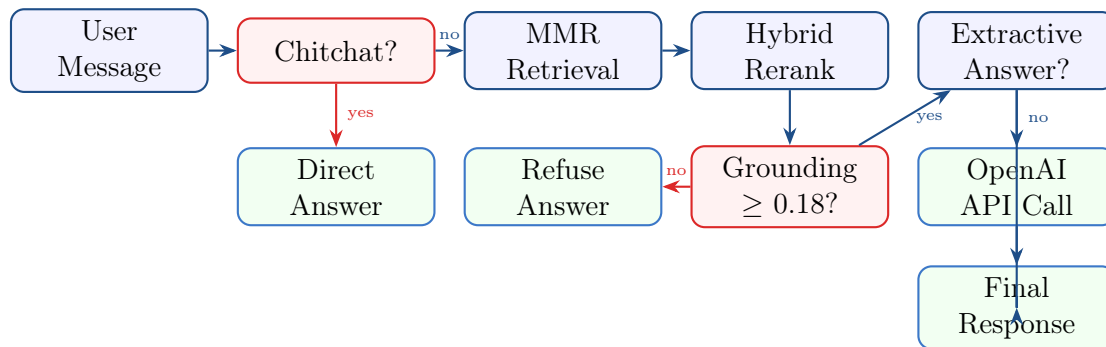


Figure 3: Retrieval and answer generation pipeline.

4.4. Persistence

Named Docker volumes (`aitutor_chroma_data`, `aitutor_uploads`) store the ChromaDB database and uploaded files independently of the source code. This ensures knowledge persists across container restarts and avoids accidental preloading from the local project folder.

5. Implementation

5.1. Hybrid Reranker

The reranker in `database.py` combines two signals: the inverse rank from MMR retrieval (semantic) and the token overlap ratio between the query and the chunk (lexical). After scoring, duplicate chunks sharing the same (`source`, `chunk_id`) pair are removed before returning the top-k results. This improves precision for keyword-heavy study material where pure semantic similarity may miss exact terms.

```

1 def _hybrid_rerank(self, query, docs, top_k):
2     q_tokens = self._tokenize(query)
  
```

```

3     scored = []
4     for rank, doc in enumerate(docs, start=1):
5         d_tokens = self._tokenize(doc.page_content)
6         lexical = len(q_tokens.intersection(d_tokens)) / max(1, len
            (q_tokens))
7         semantic = 1.0 / rank
8         score = 0.7 * semantic + 0.3 * lexical
9         scored.append((score, doc))
10    scored.sort(key=lambda x: x[0], reverse=True)
11    # Deduplicate by (source, chunk_id)
12    unique, seen = [], set()
13    for _, doc in scored:
14        key = (doc.metadata.get("source"), doc.metadata.get("
            chunk_id"))
15        if key not in seen:
16            seen.add(key)
17            unique.append(doc)
18        if len(unique) >= top_k:
19            break
20    return unique

```

Listing 1: Hybrid reranking score in database.py

5.2. Grounding Check

Before calling the LLM, `tutor_engine.py` first strips source-header lines from the context via `_extract_clean_context()`, then computes term overlap between the query and the cleaned context. If the score falls below 0.18, the system returns a refusal message. Note that an extractive answer is always attempted first; the grounding check only runs if the extractive step returns nothing.

```

1 def _grounding_strength(self, question, context):
2     clean_context = self._extract_clean_context(context)
3     if not clean_context:
4         return 0.0
5     q_terms = {t for t in re.findall(r"\w+", question.lower()) if
        len(t) > 2}
6     c_terms = {t for t in re.findall(r"\w+", clean_context.lower())
        if len(t) > 2}
7     if not q_terms or not c_terms:
8         return 0.0
9     return len(q_terms.intersection(c_terms)) / max(1, len(q_terms)
        )

```

Listing 2: Grounding strength check in tutor_engine.py

5.3. Supported File Formats

- `.txt`, `.md`: direct UTF-8 text read.
- `.pdf`: page text extraction via PyPDF2.

- `.csv`: rows flattened to pipe-separated text lines.
- `.json`: nested objects recursively flattened to `key: value` lines.

6. Evaluation Framework

6.1. Golden Dataset

A JSONL file (`golden_dataset.jsonl`) contains evaluation samples, each with a **question**, a reference **answer**, and the expected **source** filename.

6.2. Metrics

Metric	Description
Hit@k	Whether the expected source file appears in the top-k retrieved chunks.
MRR	Mean Reciprocal Rank: measures how highly the expected source is ranked.
Token F1	Lexical overlap (precision/recall) between generated and reference answers.
Semantic Similarity	Cosine similarity between embeddings of generated and reference answers.

These metrics together assess both *retrieval quality* (Hit@k, MRR) and *generation quality* (Token F1, Semantic Similarity), providing a complete picture of system performance.

Sample Evaluation Output

Samples: 10 Hit@1: 0.80 Hit@3: 0.90 MRR: 0.843
 Answer F1 (avg): 0.61 Semantic Similarity (avg): 0.74

7. Discussion: Mimicking vs. Understanding

A key question raised in the course is how far chatbot techniques can go, and what distinguishes mimicking from actual understanding.

This system retrieves relevant text chunks and selects sentences with high keyword overlap. It does not parse meaning, draw inferences, or build a conceptual model of the material. In that sense it is firmly on the *mimicking* side: it pattern-matches queries to stored text.

However, the grounding mechanism adds a layer of epistemic caution: the system refuses to answer when evidence is weak, which is closer to how a careful human would behave under uncertainty. The LLM component adds fluency and some degree of paraphrase, but the underlying knowledge still comes entirely from retrieved text.

This illustrates a practical boundary: RAG systems can produce useful, grounded responses without understanding, but they cannot reason beyond what is explicitly present in the retrieved context.

8. Known Limitations

- Scanned PDFs without a text layer return empty content; OCR is not supported.
- Semantic evaluation is embedding-based only; an LLM-judge could provide deeper quality assessment.
- Several LangChain APIs used are deprecated and should be upgraded.
- DOCX and other formats are not yet supported.
- The `generate_quiz()` function has no `max_tokens` limit, risking unbounded API cost.
- Uploaded filenames are not sanitized before use in file paths, creating a potential path traversal vulnerability.

9. Conclusion

This project successfully implements a RAG-based educational assistant that extends the course concept of conversational AI by adding hybrid retrieval reranking, grounding checks, persistent memory, and a quantitative evaluation framework. All main objectives were met: the system provides grounded answers, persists knowledge across restarts, allows file-level memory management, and measures its own performance.

The discussion of mimicking vs. understanding reveals that the system is effective precisely because it avoids hallucination through grounding, not because it understands the material. This is both its strength and its fundamental limitation.

Future work includes OCR support for scanned PDFs, DOCX ingestion, LLM-judge evaluation, and upgrading vulnerable dependencies.

Appendix: Key Implementation Components

Document Ingestion

```

1 def process_and_store_file(self, file_path):
2     filename = os.path.basename(file_path)
3     raw_text = self._load_file_text(file_path)
4     if not raw_text:
5         return 0
6     base_doc = Document(page_content=raw_text,
7                         metadata={"source": filename, "page": 1})
8     chunks = self.text_splitter.split_documents([base_doc])
9     ids = []
10    for idx, chunk in enumerate(chunks):
11        chunk.metadata["chunk_id"] = idx
12        stable = re.sub(r"[^a-zA-Z0-9_.-]", "_", filename.lower())
13        ids.append(f"{stable}::{chunk}::{idx}")
14    self.vector_store.add_documents(chunks, ids=ids)
15    return len(chunks)

```

Listing 3: Chunking and storing documents in database.py

Evaluation Pipeline

```

1 def _token_f1(prediction, reference):
2     pred_tokens = _normalize(prediction)
3     ref_tokens = _normalize(reference)
4     common = sum(min(pred_tokens.count(t), ref_tokens.count(t))
5                  for t in set(pred_tokens))
6     if common == 0:
7         return 0.0
8     precision = common / len(pred_tokens)
9     recall = common / len(ref_tokens)
10    return 2 * precision * recall / (precision + recall)
11
12 def _semantic_similarity(prediction, reference):
13     embedder = _get_semantic_embedder()
14     vectors = embedder.embed_documents([prediction, reference])
15     return _cosine_similarity(vectors[0], vectors[1])

```

Listing 4: Token F1 and semantic similarity in evaluation.py

Response Generation

```

1 def get_response(self, user_input, record_memory=True):
2     if self._is_chitchat(user_input):
3         return self._chitchat_answer(user_input)
4
5     context = self._retrieve_context(user_input)
6     grounding = self._grounding_strength(user_input, context)
7

```

```
8      # Extractive answer is attempted first
9      if context.strip():
10         extractive = self._extractive_answer(user_input, context)
11         if extractive:
12             return extractive
13
14      # Grounding check before calling the LLM
15      if context.strip() and grounding < 0.18:
16         return ("I don't have enough evidence in the retrieved
17                context "
18                "to answer confidently.")
19
20      response = self.client.chat.completions.create(
21         model=config.OPENAI_MODEL,
22         temperature=config.OPENAI_TEMPERATURE,
23         max_tokens=config.OPENAI_MAX_OUTPUT_TOKENS,
24         messages=[
25             {"role": "system", "content": "You are a precise
26                educational tutor."},
27             {"role": "user", "content": self._build_prompt(
28                user_input, context)},
29         ],
30     )
31     return self._trim_to_two_sentences(
32         response.choices[0].message.content.strip())
```

Listing 5: Grounded response logic in tutor_engine.py